

Design and Analysis of Algorithms

The Asymptotic Notations :

When analyzing an algorithm, we use three primary notations to represent the best, worst, Average case.

i. **Big-O Notation : The upper Bound:**

→ The absolute worst case scenario.

It guarantees that an algorithm will never take longer than this curve.

→ $f(n) = O(g(n))$ if there exists positive constants C and n_0 such that

$$f(n) \leq C g(n) \text{ for all } n \geq n_0$$

ii. **Big - Omega notation** : The lower bound.

→ The absolute best-case Scenario.

It guarantees that the algorithm will take atleast this much time.

→ $f(n) = \Omega(g(n))$ if there exists positive constants c and n_0 such that,

$$f(n) \geq c g(n) \quad \text{for all } n \geq n_0$$

iii. **Big - Theta notation** : the tight bound :

→ The average / precise behaviour. The algorithm grows at exactly this rate.

(Trapped between upper and lower bound of same function)

→ $f(n) = \Theta(g(n))$ if there exists positive constants C and n_0 such that:

$$C_1 g(n) \leq f(n) \leq C_2 g(n) \text{ for all } n \geq n_0$$

Common time complexities:

Notation	Name	How it Behaves
$O(1)$	Constant Time	Execution time stays exactly the same, regardless of input size.
$O(\log n)$	Logarithmic Time	The problem size is cut in half with each step.
$O(n)$	Linear Time	Execution time grows in direct proportion to input size.
$O(n \log n)$	Linearithmic Time	Usually occurs when you split a problem in half, solve both, and merge them.
$O(n^2)$	Quadratic Time	Often involves nested loops over the data.
$O(2^n)$	Exponential Time	Growth doubles with each addition to the input dataset.
$O(n!)$	Factorial Time	The slowest growth rate; completely stalls on even medium-sized inputs.
Fastest → Slowest		

Substitution method:

- ① Guess the form of solution.
- ② prove by mathematic Induction.

1. Solve the recurrence relation using Substitution method.

$$T(n) = 2T(n/2) + n$$

Let's assume $T(1) = 1$

Guess: $T(n) \leq c n \log n$, $c > 0$

For $n=2$

$$T(n) \leq c n \log n$$

$$2T(2/2) + 2 \leq c 2 \log 2 \quad / \log_2 2 = 1$$

$$2T(1) + 2 \leq 2c \quad / T(1) = 1$$

$$4 \leq 2c \quad \text{where } c \geq 2$$

$$n=4$$

$$T(n) \leq c n \log n$$

$$2T(n/2) + n \leq c n \log n$$

$$2T(4/2) + 4 \leq c 4 \log 4 \quad / \log_2 4 = 2$$

$$2T(2) + 4 \leq 8c$$

$$2(4) + 4 \leq 8c \quad / T(2) = 4$$

$$12 \leq 8c \quad \text{where } c \geq 2$$

By Mathematical Induction,

$$T(n) \leq cn \log n$$

$$T(n) \leq 2T(n/2) + n$$

$$T(n) \leq 2 \left(\frac{cn}{2} \log n/2 \right) + n$$

$$T(n) \leq cn \log n/2 + n$$

$$T(n) \leq cn \log n - \cancel{cn \log 2}^1 + n$$

$$T(n) \leq cn \log n - cn + n$$

$$T(n) \leq cn \log n, \quad c \geq 1$$

Iteration / Back-substitution method:

- ① Expand: write out the recurrence then substitute it into itself multiple times.
- ② Find the pattern: Generalize the expansion in terms of k .
- ③ substitute the base case: solve for k to reach the base case, replace it, calculate the total summation.

2. Solve the recurrence relation using iteration method :

$$T(n) = T(n/2) + 1$$

Let's assume: $T(1) = 1$

$$T(n) = T(n/2) + 1$$

If $n = n/2$ then,

$$T(n/2) = T(n/4) + 1$$

If $n = n/4$ then,

$$T(n/4) = T(n/8) + 1$$

$$T(n) = T(n/2) + 1$$

$$T(n) = T(n/4) + 1$$

$$T(n) = T(n/2)$$

$$T(n) = T(n/8) + 1$$

$$T(n) = T(n/4)$$

$\vdots$

$\vdots$

$$T(n) = T(n/2^k) + k$$

$$\frac{n}{2^k} = 1$$

$$n = 2^k$$

$$k = \log_2 n$$

$$T(n) = T(1) + \log_2 n$$

$$T(n) = 1 + \log_2 n$$

$$/ T(1) = 1$$

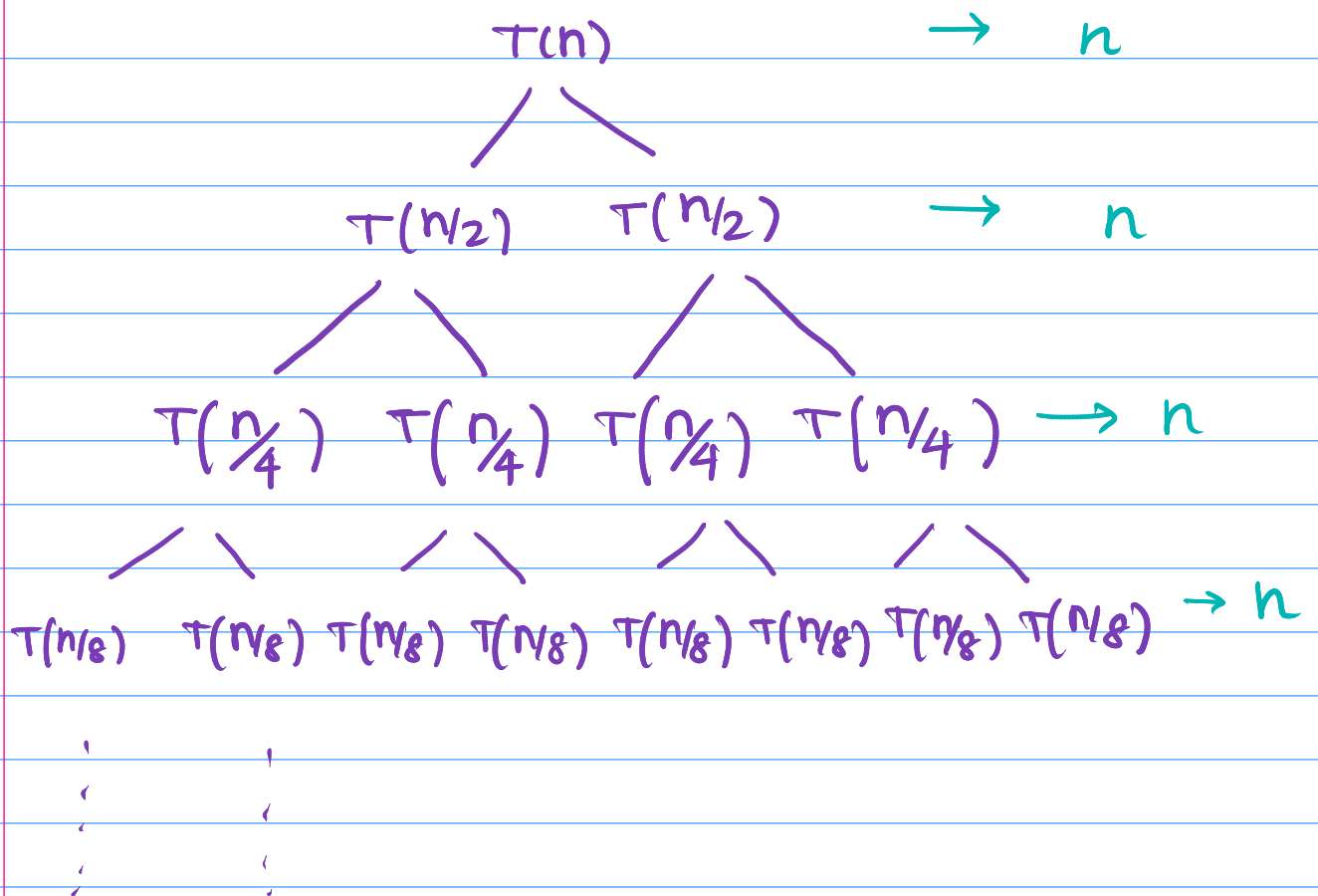
$$T(n) = O(\log_2 n)$$

Recursive tree :

- ① Draw the tree: Branch the recurrence down until we reach the base case (problem size 1).
- ② Sum of cost at each level: Add up the non-recurrence work done by all the nodes in a single row.
- ③ Sum all costs: Add up the costs of every single level to find the total time complexity.

3. Solve the recurrence relation by using tree method:

$$T(n) = 2T(n/2) + cn$$



Till problem size 1 after k levels.

At level k ,

There are 2^k subproblems of size $\frac{n}{2^k}$

$$\text{Cost} = 2^k \times \frac{cn}{2^k}$$

$$\text{Cost of levels} = cn.$$

Finding depth,

Let k be the bottom most level, then

$$\frac{n}{2^k} = 1.$$

$$\boxed{k = \log n}$$

∴ Total levels in the tree including base case is $\log n + 1$.

Summing all,

$$\text{Total cost} = \sum_{i=0}^{\log n} (\text{cost of level } k)$$

$$\text{Total cost} = \sum_{i=0}^{\log n} (n + n + n + \dots + n)$$

$$\text{Total cost} = cn \times (\log n + 1)$$

$$\text{Total cost} = cn \log n + cn$$

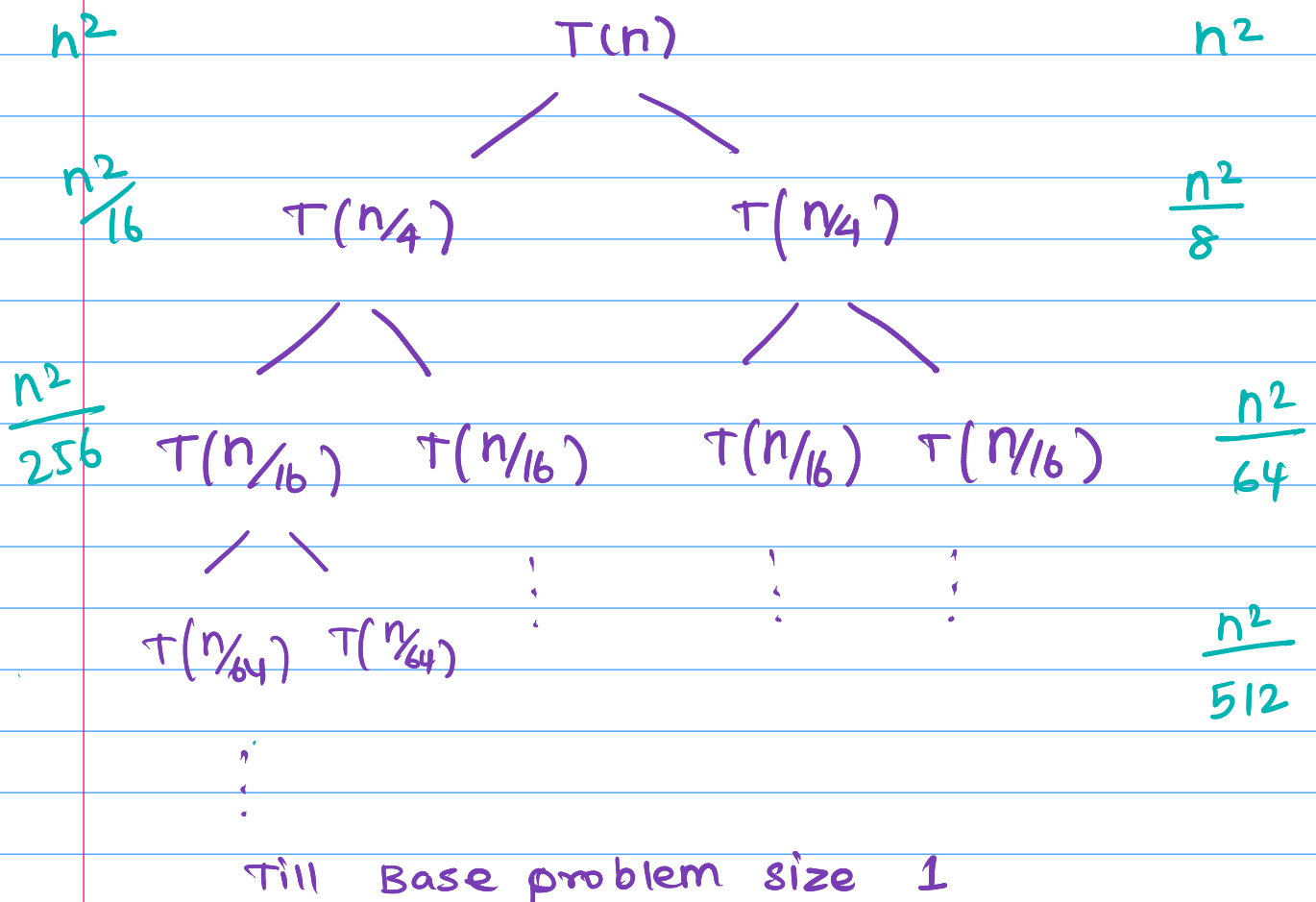
$$T(n) \leq O(n \log n)$$

4. Solve the recurrence relation using

Tree method:

$$T(n) = 2T(n/4) + cn^2$$

Let's assume $T(1) = 1$



At level k ,

There are 2^k subproblems for each k level,

$$\text{cost} = 2^k \times \frac{cn^2}{(4^k)^2}$$

$$\text{cost} = \frac{cn^2}{8^k}$$

Finding depth,

Let k be the most bottom level then

$$\frac{n}{4^k} = 1$$

$$n = 4^k$$

$$\boxed{k = \log_4 n}$$

$$\text{leaves} = 2^k = 2^{\log_4 n}$$

$$= n^{\log_4 2}$$

$$\text{no. of leaves} = n^{1/2}$$

If $T(n) = O(n)$ then

$$\text{Leaf cost} = O(\sqrt{n})$$

Summing up all,

$$\text{Total cost} = \sum_{i=0}^{\log_4 n - 1} (\text{cost at each level } i)$$

$$\text{Total cost} = \sum_{i=0}^{\log_4 n - 1} \frac{cn^2}{8^i} + O(\sqrt{n})$$

$$\text{Total cost} = cn^2 \sum_{i=0}^{\log_4 n - 1} \frac{1}{8^i} + O(\sqrt{n})$$

$$= cn^2 \left(\frac{1}{1 - 1/8} \right) = 8/7$$

$$T(n) \leq \frac{8cn^2}{7} + O(\sqrt{n})$$

$$T(n) \leq O(n^2)$$

1. Case 1: $r < 1$ (The Root-Heavy Case)

- **What is happening:** The work decreases exponentially as you go down the tree. The root level does almost all the heavy lifting.
- **The Formula:**

$$T(n) = \Theta(\text{Root Cost})$$

$$T(n) = \Theta(f(n))$$

- **Proof summation technique:** Uses the infinite geometric series upper bound:

$$\sum_{i=0}^{\infty} r^i = \frac{1}{1-r}$$

- **Example:** $T(n) = 3T(n/4) + cn^2 \implies r = \frac{3}{16} < 1 \implies T(n) = \Theta(n^2)$

2. Case 2: $r = 1$ (The Evenly Distributed Case)

- **What is happening:** Every single level of the recursion tree does the exact same amount of work.
- **The Formula:**

$$T(n) = \Theta(\text{Cost of One Level} \times \text{Height of Tree}) + \Theta(\text{Leaf Cost})$$

Since the level cost and the leaf cost have the same growth rate here, it simplifies beautifully to:

$$T(n) = \Theta(f(n) \cdot \log_b n)$$

- **Example:** $T(n) = 2T(n/2) + cn \implies r = 1 \implies T(n) = \Theta(n \log n)$

3. Case 3: $r > 1$ (The Leaf-Heavy Case)

- **What is happening:** The work increases exponentially as you go down the tree. The bottom level (the leaves) does almost all the work. [Wikipedia](#)
- **The Formula:**

$$T(n) = \Theta(\text{Leaf Cost})$$

$$T(n) = \Theta(n^{\log_b a})$$

- **Proof summation technique:** Uses the **finite geometric series** formula:

$$\sum_{i=0}^k r^i = \frac{r^{k+1} - 1}{r - 1}$$

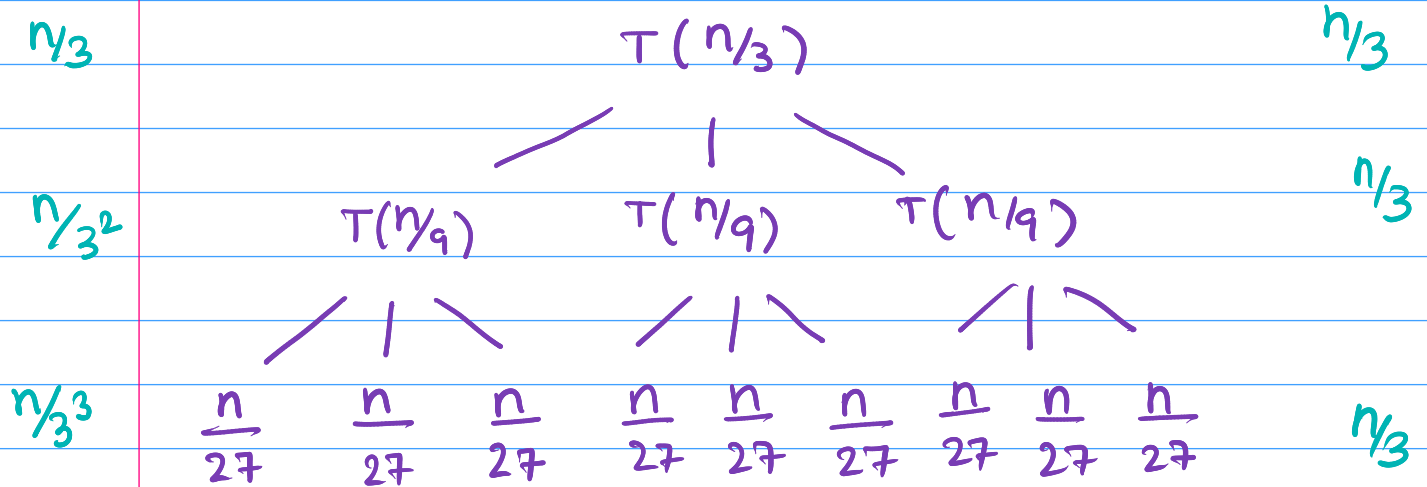
- **Example:** $T(n) = 3T(n/2) + cn \implies r = \frac{3}{2} > 1 \implies T(n) = \Theta(n^{\log_2 3})$

Ratio Value	Where is the work?	Total Time Complexity $T(n)$
$r < 1$	Root (Top)	$\Theta(f(n))$
$r = 1$	Even (All levels)	$\Theta(f(n) \log_b n)$
$r > 1$	Leaves (Bottom)	$\Theta(n^{\log_b a})$

5. Solve the recurrence relation using Tree method:

$$T(n) = 3T(n/3) + cn$$

Let's assume $T(1) = 1$



Till the base condition $T(1) = 1$

At level k ,

There are 3^k sub problems then

$$\text{cost} = 3^k \times \frac{cn}{3^k}$$

$$\text{cost} = cn.$$

At the depth,

$$\frac{n}{3^k} = 1$$

$$n = 3^k$$

$$k = \log_3 n$$

$$\text{leaves } 3^k = 3^{\log_3 n}$$

$$\text{leaves} = n^{\log_3 3}$$

$$\text{Leaf IF } T(1) = O(1)$$

$$\text{leaf cost} = O(n)$$

$$\text{Total cost} = \sum_{i=0}^{\log_3 n + 1} (\text{cost at each level } i)$$

$$\text{If } r=1, \text{ then } T(n) = O(f(n) \cdot \log_b n)$$

$$T(n) = O(n \log_3 n)$$

6. Solve the recurrence relation by

Tree method:

$$T(n) = 2T(n/2) + cn^2$$

$$\text{Leaves} = 2^k = 2^{\log n} = n^{\log 2}$$

$$\text{leaves} = n$$

If $O(1) = T(1)$ then leaf cost = $O(n)$

Summing up all,

$$\text{Total cost} = \sum_{i=0}^{\log n - 1} \frac{n^2}{2^i} + O(n)$$

$$= n^2 \sum_{i=0}^{\log n - 1} \frac{1}{2^i} + O(n)$$

$$= 2n^2 + O(n)$$

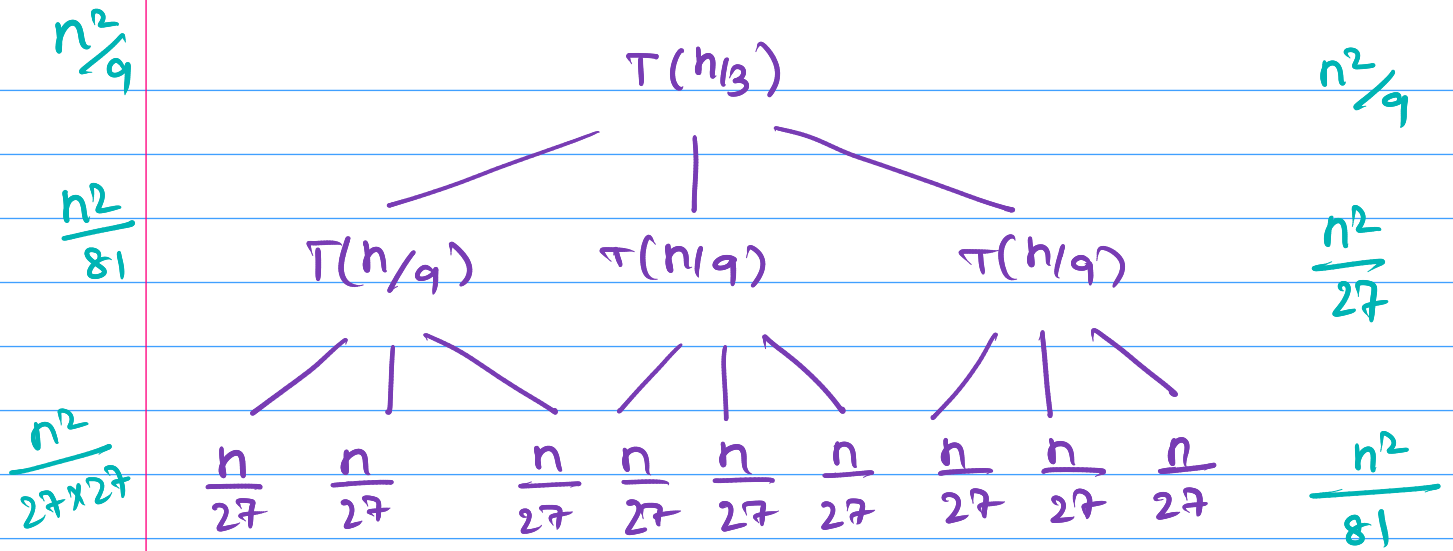
$$T(n) \leq O(n^2)$$

7. Solve the recurrence relation using

Tree method.

$$T(n) = 3T(n/3) + cn^2$$

Assume $T(1) \leq 1$,



Till the base size 1 $T(1) = 1$

At the level k ,

The problem divides into 3^k subproblems,

$$\text{level cost} = 3^k \times \frac{n^2}{9^k} = \frac{n^2}{3^k}$$

At the depth,

$$\text{Height} \Rightarrow \frac{n}{3^k} = 1 \quad \boxed{k = \log_3 n}$$

$$\text{Leaf} \Rightarrow 3^k = 3^{\log_3 n} = n$$

$$\text{Leaf cost} = O(n)$$

Summing up all cost,

$$\text{Total cost} = \sum_{i=0}^{\log_3 n - 1} \frac{n^2}{3^i} + o(n)$$

$$= n^2 \sum_{i=0}^{\log_3 n - 1} \frac{1}{3^i} + o(n)$$

$$= \frac{3}{2} n^2 + o(n)$$

$$T(n) \leq O(n^2)$$

Master method:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

i. If $f(n) = \Theta(n^{\log_b a - \epsilon})$, $\epsilon > 0$,

$$\text{then } T(n) = \Theta(n^{\log_b a})$$

ii. If $f(n) = \Theta(n^{\log_b a})$, $\epsilon = 0$, then

$$T(n) = \Theta(n^{\log_b a} \lg n)$$

iii. If $f(n) = \Theta(n^{\log_b a + \epsilon})$, $\epsilon > 0$,

then $T(n) = \Theta(f(n))$.

if $a f\left(\frac{n}{b}\right) \leq c f(n)$, $0 < c < 1$.

1. $T(n) = 9T\left(\frac{n}{3}\right) + n$

$$a=9 \quad b=3 \quad f(n)=n$$

$$f(n) = \Theta(n^{\log_b a - \epsilon})$$

$$f(n) = n$$

$$n = n^{\log_b a - \epsilon}$$

$$n = n^{\log_3 9 - \epsilon}$$

$$n = n^{2 - \epsilon}$$

$$2 - \epsilon = 1$$

$$\boxed{\epsilon = 1}$$

$$\text{so } T(n) = \Theta(n^{\log_b a})$$

$$T(n) = \Theta(n^2)$$

$$2. \quad T(n) = T\left(\frac{2n}{3}\right) + 1$$

$$a=1 \quad b=3/2 \quad f(n)=1$$

$$f(n) = \Theta(n^{\log_b a - \epsilon})$$

$$f(n) = 1$$

$$1 = n^{\log_b a - \epsilon}$$

$$1 = n^{\log_{3/2} 1 - \epsilon}$$

$$1 = n^{0 - \epsilon}$$

$$\epsilon = 0.$$

$$T(n) = \Theta(n^{\log_b a} \lg n)$$

$$T(n) = \Theta(n^0 \log n)$$

$$T(n) = \Theta(\log n).$$

3. $T(n) = 3T(n/4) + n \log n.$

$$a = 3 \quad b = 4 \quad f(n) = n \log n$$

$$f(n) = \Theta(n^{\log_b a - \epsilon})$$

$$f(n) = n \log n.$$

When comparing $n \times \log n$, n is bigger so,

$$n = n^{\log_b a - \epsilon}$$

$$n = n^{\log_4 3 - \epsilon}$$

$$n = n^{0.79 - 0.2}$$

$$\boxed{\epsilon = 0.2}$$

$$\text{so, } T(n) = \Theta(f(n))$$

$$T(n) = \Theta(n \log n)$$

Module 2

Divide and Conquer:

1. max subset sum
2. Strasson's matrix multiplication

Max Subarray problem:

{ -1, 3, -2, 5, -6, 4, 2, -1 }

-1	3	-2	5		-6	4	2	-1
Left subarray					Right subarray			
5		(5)			-6		(-6)	
5-2		(3)			-6+4		(-2)	
5-2+3		(6)	✓		-6+4+2		(0)	✓
5-2+3-1		(5)			-6+4+2-1		(-1)	

Leftmax = 6

Rightmax = 0

$$\text{Cross Sum} = \text{leftsum} + \text{RightSum}$$

$$\text{cross sum} = 6 + 0$$

$$\text{Cross sum} = 6$$

Left Subproblem:

$$\{-1, 3, -2, 5\}$$

$$-1 = -1$$

$$-1 + 3 = 2$$

$$-1 + 3 - 2 = 0$$

$$-1 + 3 - 2 + 5 = 5$$

$$3 = 3$$

$$3 - 2 = 1$$

$$3 - 2 + 5 = 6 \checkmark$$

$$-2 = -2$$

$$-2 + 5 = 3$$

$$5 = 5$$

Left subproblem maximum = 6

$\{3, -2, 5\}$

Right subproblem:

$\{-6, 4, 2, -1\}$

$$-6 = -6$$

$$-6 + 4 = -2$$

$$-6 + 4 + 2 = 0$$

$$-6 + 4 + 2 - 1 = -1$$

$$4 = 4$$

$$4 + 2 = \textcircled{6} \checkmark$$

$$4 + 2 - 1 = 5$$

$$2 = 2$$

$$2 - 1 = 1$$

$$-1 = -1$$

Right subproblem sum = 6 $\{4, 2\}$

Brute-Force Technique

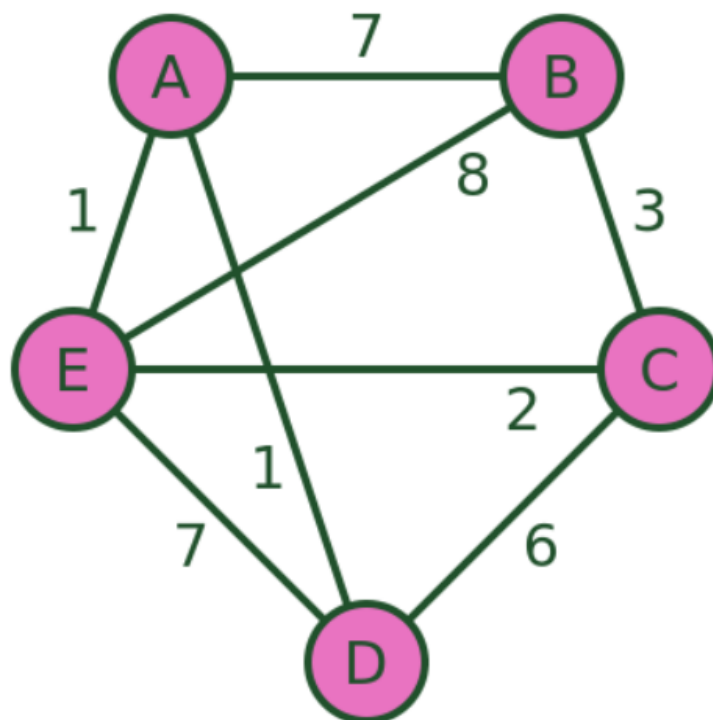
- ① Max and Min in Array
- ② Travelling Salesman problem.

Travelling Salesman problem:

$$\text{Total possible routes} = \frac{(n-1)!}{2}$$

$n \rightarrow$ no. of cities for complete graph

$\Rightarrow$ Calculate the minimum cost and path for the given graph using Brute force technique.



Making matrix from Routes and cost

	A	B	C	D	E
A	0	7	∞	1	1
B	7	0	3	∞	8
C	∞	3	0	6	2
D	1	∞	6	0	7
E	1	8	2	7	0

A - B 7

A - D 1

A - E 1

B - C 3

B - E 8

C - D 6

C - E 2

D - E 7

$\therefore$ No direct

connection between

A and C nor

between B and D.

$A \neq C$
 $B \neq D$

Let's keep A as the starting point

$A \rightarrow D \rightarrow C \rightarrow B \rightarrow E \rightarrow A$ 19 ✓

$A \rightarrow D \rightarrow E \rightarrow C \rightarrow B \rightarrow A$ 20

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow A$ 24

$A \rightarrow D \rightarrow C \rightarrow E \rightarrow B \rightarrow A$ 24

Minimum cost : 19

Shortest path: $A \rightarrow D \rightarrow C \rightarrow B \rightarrow E \rightarrow A$

Total routes in Incomplete graph:

$$(n-1-k)! \cdot 2^{k-2}$$

$n \rightarrow$ no. of vertex
 $k \rightarrow$ missing edges

using PIE / principle of Inclusion -

Exclusion method :

No. of routes that don't use AC

pair A and C together so,

(AC), B, D, E

paired term $k=1$

$$(n-1-k)! 2^{k-2} \Rightarrow (5-1-1)! 2^{1-2}$$

$$3! 2^{-1}$$

$$\frac{6}{2} = 3 \text{ routes}$$

similarly B and D 3 routes.

AC and BD together so (AC), (BD), E

$$(n-1-k)! 2^{k-1} \Rightarrow (5-1-2)! 2^{2-2}$$

$$2! \times 1 = 2 \text{ routes}$$

Invalid Routes = $3+3+2 = 8$ routes.

Valid Routes = Total Routes - Invalid routes

$$\text{Valid Routes} = 12 - 8 = 4 \text{ Routes}$$